Partie 1
Apprentissage Non supervisé



Apprentissage Non supervisé

- L’apprentissage non supervisé consiste à analyser et structurer des données sans avoir recours à des étiquettes ou des sorties pré-définies.

- **Il permet d’ :**
  - Identifier des structures ou des relations cachées.
  - Explorer des données complexes.

- **Exemples d’applications :**
  - Regroupement de clients.
  - Réduction de dimensions pour visualisation.
  - Détection d’anomalies dans les systèmes industriels.

Apprentissage Non supervise: Concepts fondamentaux



Caractéristiques des données :

Données brutes sans étiquettes.

Similarité entre les observations basée sur des métriques (distance Euclidienne, Manhattan, Cosine, etc.).



Approches clés :

Clustering : Groupement d'éléments similaires.

Réduction de dimensions : Simplification de la représentation tout en conservant l'information importante.

Détection d'anomalies : Identification de points atypiques dans les données.



Enjeux et limites :

Nécessité de choisir un nombre optimal de clusters ou dimensions.

Problèmes d'interprétabilité.

Clustering

- Le clustering consiste à regrouper les données en clusters ou groupes homogènes.
- **Algorithmes populaires :**
  - **K-Means :**
    - Partitionne les données en k clusters en minimisant la variance intra-cluster.
    - Exemples visuels pour montrer la convergence des centres de cluster.
  - **Clustering Hiérarchique :**
    - Regroupe les données en niveaux hiérarchiques représentés par un dendrogramme.
  - **DBSCAN :**
    - Détecte des clusters de formes arbitraires et traite bien les anomalies.

Réduction de Dimensionnalité

- Réduire le nombre de variables tout en conservant un maximum d'information.
- **Algorithmes populaires :**
  - **PCA (Analyse en Composantes Principales) :**
    - Projette les données sur des axes orthogonaux en maximisant la variance.
    - Exemple visuel montrant des données projetées sur les deux premiers axes.
  - **t-SNE :**
    - Méthode de réduction non linéaire pour la visualisation.
    - Application sur le dataset MNIST.
  - **UMAP :**
    - Meilleure préservation des distances globales et locales.

Détection d’Anomalies

- Identifier des données qui diffèrent significativement de la distribution normale.
- **Algorithmes populaires :**
  - **Isolation Forest :**
    - Isoler des points en utilisant des arbres de décision aléatoires.
  - **Méthodes basées sur la distribution :**
    - Analyse des valeurs extrêmes selon la distribution statistique.

Métriques d’Évaluation

- **Clustering :**
  - Indice de silhouette : mesure la cohésion et la séparation des clusters.
  - Rand Index : compare les clusters prédits aux vrais clusters.
- **Réduction de dimension :**
  - Conservation de la variance totale.
- **Détection d’anomalies :**
  - Précision, rappel, F1-score, et taux de faux positifs/négatifs.

Clustering

- Objectif = structuration des données
- On cherche à regrouper les points proches/similaires en « paquets ou groupes ou classes»
- La qualité d'un regroupement dépend donc de la mesure de similarité utilisée par la méthode et de son implémentation
- Pb : les groupes peuvent être assez bien définis et séparés, ou au contraire imbriqués/sans frontières claires, et de formes quelconques



Clustering

Domaines d'application

Le Clustering s'applique dans plusieurs domaines

Domaine	Forme des données	Clusters
Text mining	Textes Mails	Textes proches Dossiers automatiques
Web mining	Textes et images	Pages web proches
BioInformatique	Gènes	Gènes ressemblants
Marketing	Infos clients, produits achetés	Segmentation de la clientèle
Segmentation d'images	Images	Zones homogènes dans l'image
Web log analysis	Clickstream	Profils utilisateurs

Clustering

- **K-Means**
- **Clustering Hiérarchique**
- **DBSCAN**

k-moyennes (K-means)



Pour résoudre certains problèmes complexes, On peut segmenter la population en espérant que le problème est alors plus simple à résoudre sur les groupes ainsi constitués.



La méthode K-moyenne est une méthode de segmentation.



Les observations sont les n-uplets de valeurs.



Définition d’une mesure de similarité pour comparer les groupes.

k-moyennes (K-means)

- Sélectionner K points pour être les centroïdes initiaux
- Tant que les centroïdes changent d'une itération à l'autre
  - Former les K segments en affectant chacun des points au centroïde le plus proche
  - Recalculer le centroïdes de chaque segment
- Caractéristiques:
  - ✓ L'idée de base de l'algorithme est très simple
  - ✓ Segmentation par partitionnement
  - ✓ Les segments sont définis par leurs prototypes
  - ✓ Le nombre de segments doit être spécifié
- Il faut choisir:
  - Une fonction distance
  - Une définition de centroïde

k-moyennes

- Initialiser les centres $\mu_1, \cdots \mu_K$
- Répeter
  - Affectation de chaque point à son cluster le plus proche

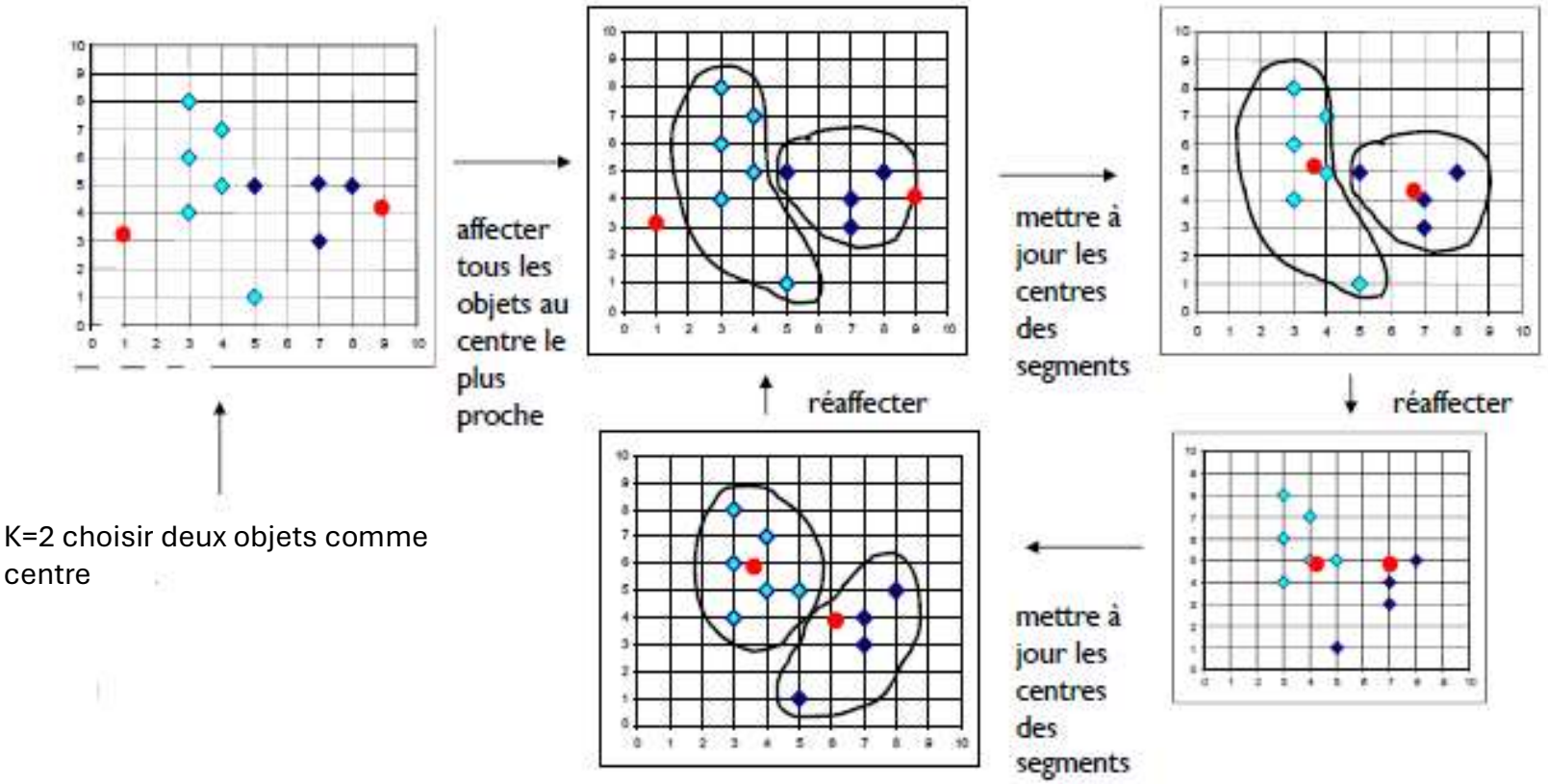
$$\mathcal{C}_\ell \leftarrow x_i \quad \text{tel que} \quad \ell = \arg \min_k d(x_i, \mu_k)$$

  - Recalculer le centre μ_k de chaque cluster

$$\mu_k = \frac{1}{N_k} \sum_{i \in \mathcal{C}_k} x_i \quad \text{avec} \quad N_k = \text{card}(\mathcal{C}_k)$$

- Tant que $\| \Delta\mu \| > \epsilon$

k- moyennes


affecter tous les objets au centre le plus proche
mettre à jour les centres des segments
réaffecter
mettre à jour les centres des segments
réaffecter
K=2 choisir deux objets comme centre

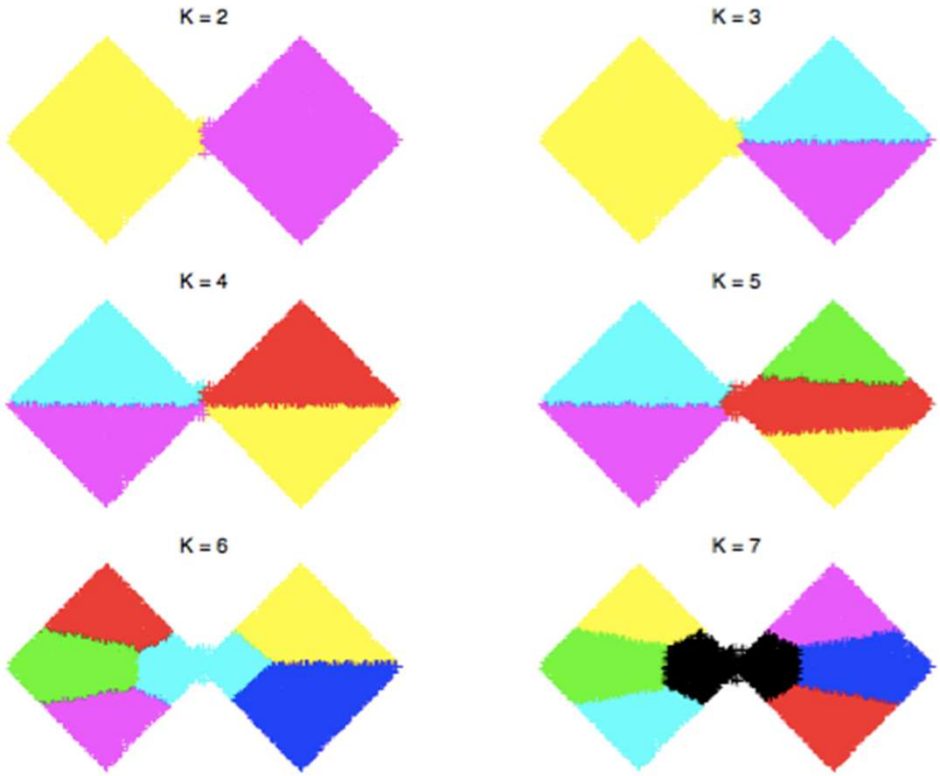
k- moyennes

- Les centroïdes initiaux peuvent varier d'un essai à l'autre
- Les segments produits peuvent différer d'un essai à l'autre
- Le centroïde est typiquement le centre des points
- La proximité est typiquement mesurée par la distance euclidienne, la similarité du cosinus, …
- L'algorithme converge en général pour les mesures typiques de similarité
- Souvent l'algorithme converge en quelques itérations
- Complexité est O(n * K * I * d)

  n = nombre de points, K = nombre de segments,

  I = nombre d'itérations, d = nombre d'attributs

k- moyennes



K-moyennes

- **Techniques pour déterminer le nombre optimal de clusters**
  1. **Elbow Method :**
     - Trace le SSE (somme des erreurs quadratiques) en fonction de k.
     - Choix du point d'inflexion ("coude").
  2. **Silhouette Score :**
     - Évalue la qualité des clusters en fonction de la séparation et de la cohésion.
     - Score entre -1 et 1 : plus il est élevé, mieux c'est.
  3. **Critère Gap :** Compare la dispersion des données avec une référence aléatoire.

Kmeans et Python

Classe	Fromages
0	CarredelEst
0	Camembert
0	Fr.chevrepatemolle
0	Chabichou
0	Chaource
0	Coulomniers
1	Petitsuisse40
1	Fr.frais40nat.
1	Fr.frais20nat.
1	Yaourtlaitent.nat.
2	Parmesan
2	Edam
2	Emmental
2	Beaufort
2	Comte
3	Tome
3	SaintPaulin
3	Rocquefort
3	Reblochon
3	Pyrenees
3	PontlEveque
3	Cheddar
3	Morbier
3	Maroilles
3	Bleu
3	Vacherin
3	Cantal
3	Babybel
3	Fr.fondu.45

```
#k-means sur les données centrées et réduites
from sklearn import cluster
kmeans = cluster.KMeans(n_clusters=4)
kmeans.fit(fromage_cr)

#index triés des groupes
idk = np.argsort(kmeans.labels_)

#affichage des observations et leurs groupes
print(pandas.DataFrame(fromage.index[idk],kmeans.labels_[idk]))
```

Kmeans et Python

```
#distances aux centres de classes des observations
print(kmeans.transform(fromage_cr))
```

```
#correspondance avec les groupes de la CAH
pandas.crosstab(groupes_cah,kmeans.labels_)
```

col_0	0	1	2	3
row_0				
1	0	4	0	0
2	4	0	0	0
3	0	0	4	0
4	2	0	1	14

Groupe	0	1	2	3
CarredelEst	2.22	5.53	5.22	2.92
Babybel	3.02	5.19	2.79	0.74
Beaufort	5.16	7.51	1.15	2.86
Bleu	3.24	6.12	3.90	2.11
Camembert	1.93	5.40	5.10	3.54
Cantal	4.02	6.30	2.20	1.19
Chabichou	1.78	5.93	4.53	3.39
Chaource	1.03	5.55	5.09	3.46
Cheddar	3.75	6.82	2.29	1.95
Comte	5.44	7.84	1.35	3.42
Coulomniers	1.96	4.84	4.75	2.49
Edam	4.23	6.13	1.34	2.25
Emmental	5.53	7.48	0.90	3.40
Fr.chevrepatemolle	3.10	5.01	7.09	5.54
Fr.fondu.45	2.84	5.28	4.45	1.89
Fr.frais20nat.	5.33	0.68	7.61	6.00
Fr.frais40nat.	4.55	1.01	7.32	5.61
Maroilles	4.20	6.46	2.45	1.53
Morbier	3.47	6.06	2.50	0.65
Parmesan	5.23	7.94	2.11	3.60
Petitsuisse40	4.38	1.21	7.13	5.40
PontlEveque	3.08	4.69	3.52	1.26
Pyrenees	3.28	5.71	2.81	0.71
Reblochon	2.74	5.31	2.94	0.81
Rocquefort	3.02	6.81	4.22	2.18
SaintPaulin	3.47	5.12	2.67	1.37
Tome	2.73	5.25	3.67	1.26
Vacherin	3.79	5.27	2.63	1.50
Yaourtlaitent.nat.	6.09	1.93	7.46	5.94

Kmeans et Python

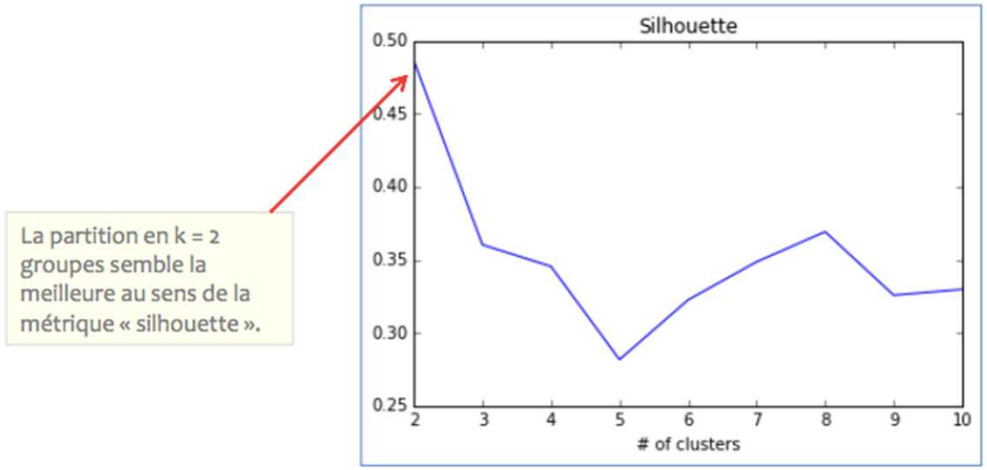
```
#librairie pour évaluation des partitions
from sklearn import metrics

#utilisation de la métrique "silhouette"
#faire varier le nombre de clusters de 2 à 10
res = np.arange(9,dtype="double")
for k in np.arange(9):
    km = cluster.KMeans(n_clusters=k+2)
    km.fit(fromage_cr)
    res[k] = metrics.silhouette_score(fromage_cr,km.labels_)

print(res)

#graphique
import matplotlib.pyplot as plt
plt.title("Silhouette")
plt.xlabel("# of clusters")
plt.plot(np.arange(2,11,1),res)
plt.show()
```

Kmeans et Python



Principe de la méthode Elbow

```
sse = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    sse.append(kmeans.inertia_)  # inertia_ = SSE
# Visualisation
plt.figure(figsize=(10, 6))
plt.plot(k_range, sse, 'bo-', markersize=8)
plt.xlabel('Nombre de clusters (k)')
plt.ylabel('SSE (Inertie)')
plt.title('Méthode Elbow pour déterminer k optimal')
plt.grid(True, alpha=0.3)
plt.xticks(k_range)
plt.show()
```

Principe de la méthode Elbow

- **Comment identifier le "coude" : (Approche visuelle)**

```
# Version avec annotation du coude
plt.figure(figsize=(10, 6))
plt.plot(k_range, sse, 'bo-', markersize=8, linewidth=2)
plt.xlabel('Nombre de clusters (k)')
plt.ylabel('SSE (Inertie)')
plt.title('Méthode Elbow - Point optimal à k=4')
plt.grid(True, alpha=0.3)

# Annotation du coude
plt.axvline(x=4, color='red', linestyle='--', alpha=0.7, label='Coude (k=4)')
plt.legend()
plt.show()
```

Principe de la méthode Elbow

- **Interprétation des résultats :**
- **k trop petit** : SSE élevé, sous-clustering
- **k optimal** : Point où l'ajout d'un cluster n'améliore plus significativement le SSE
- **k trop grand** : Diminution marginale du SSE, risque de sur-apprentissage

K-means



Forces :

Simple et rapide.

Efficace pour les données sphériques.



Limites :

Nombre de clusters k à fixer à l’avance.

Sensible aux valeurs aberrantes.

Exercice K-means

Vous disposez du jeu de données suivant (8 points en 2 dimensions) :

$$\text{Données} : \{(1,1), (2,1), (4,3), (5,4), (3,4), (6,5), (8,7), (7,8)\}$$

1. Initialisez **aléatoirement** deux centres de clusters C_1 et C_2 :
   - $C_1 = (1,1)$
   - $C_2 = (7,8)$

Exercice K-means

2. **Étape 1 : Affectation des points aux clusters**
   - Calculez la **distance euclidienne** entre chaque point et les deux centres de clusters.
   - Affectez chaque point au cluster le plus proche.
3. **Étape 2 : Mise à jour des centres de clusters**
   - Recalculez les nouveaux centres de clusters comme la moyenne des points attribués à chaque cluster :

$$C_1 = \frac{\text{Somme des points du cluster 1}}{\text{Nombre de points du cluster 1}}$$

$$C_2 = \frac{\text{Somme des points du cluster 2}}{\text{Nombre de points du cluster 2}}$$

4. Répétez les étapes 1 et 2 jusqu'à ce que les centres ne changent plus.

Correction

Calculs :

Point	Distance à $C_1 = (1,1)$	Distance à $C_2 = (7,8)$	Cluster assigné
$(1,1)$	$\sqrt{(1-1)^2+(1-1)^2} = 0$	$\sqrt{(7-1)^2+(8-1)^2} = 8.49$	C_1
$(2,1)$	$\sqrt{(2-1)^2+(1-1)^2} = 1$	$\sqrt{(7-2)^2+(8-1)^2} = 8.06$	C_1
$(4,3)$	$\sqrt{(4-1)^2+(3-1)^2} = 3.61$	$\sqrt{(7-4)^2+(8-3)^2} = 5.83$	C_1
$(5,4)$	$\sqrt{(5-1)^2+(4-1)^2} = 5$	$\sqrt{(7-5)^2+(8-4)^2} = 4.47$	C_2
$(3,4)$	$\sqrt{(3-1)^2+(4-1)^2} = 3.61$	$\sqrt{(7-3)^2+(8-4)^2} = 5.66$	C_1
$(6,5)$	$\sqrt{(6-1)^2+(5-1)^2} = 6.40$	$\sqrt{(7-6)^2+(8-5)^2} = 3.16$	C_2
$(8,7)$	$\sqrt{(8-1)^2+(7-1)^2} = 9.22$	$\sqrt{(7-8)^2+(8-7)^2} = 1.41$	C_2
$(7,8)$	$\sqrt{(7-1)^2+(8-1)^2} = 9.22$	$\sqrt{(7-7)^2+(8-8)^2} = 0$	C_2

Correction

Résultats de l'affectation (Itération 1) :

- Cluster C_1 : $\{(1,1), (2,1), (4,3), (3,4)\}$
- Cluster C_2 : $\{(5,4), (6,5), (8,7), (7,8)\}$

$$C_1 = \frac{1}{n}\sum_{i=1}^{n}(x_i, y_i), \quad C_2 = \frac{1}{m}\sum_{j=1}^{m}(x_j, y_j)$$

- Nouveau C_1 :

$$C_1 = \frac{(1,1) + (2,1) + (4,3) + (3,4)}{4} = \left(\frac{10}{4}, \frac{9}{4}\right) = (2.5, 2.25)$$

- Nouveau C_2 :

$$C_2 = \frac{(5,4) + (6,5) + (8,7) + (7,8)}{4} = \left(\frac{26}{4}, \frac{24}{4}\right) = (6.5, 6)$$

Correction

Itération 2 : Affectation des points aux clusters

Calculs avec les nouveaux centres :

Point	Distance à $C_1 = (2.5, 2.25)$	Distance à $C_2 = (6.5, 6)$	Cluster assigné
$(1, 1)$	$\sqrt{(2.5-1)^2 + (2.25-1)^2} = 2.08$	$\sqrt{(6.5-1)^2 + (6-1)^2} = 7.43$	C_1
$(2, 1)$	$\sqrt{(2.5-2)^2 + (2.25-1)^2} = 1.30$	$\sqrt{(6.5-2)^2 + (6-1)^2} = 6.40$	C_1
$(4, 3)$	$\sqrt{(4-2.5)^2 + (3-2.25)^2} = 1.60$	$\sqrt{(6.5-4)^2 + (6-3)^2} = 4.27$	C_1
$(5, 4)$	$\sqrt{(5-2.5)^2 + (4-2.25)^2} = 3.20$	$\sqrt{(6.5-5)^2 + (6-4)^2} = 2.50$	C_2
$(3, 4)$	$\sqrt{(3-2.5)^2 + (4-2.25)^2} = 1.87$	$\sqrt{(6.5-3)^2 + (6-4)^2} = 4.27$	C_1
$(6, 5)$	$\sqrt{(6-2.5)^2 + (5-2.25)^2} = 4.70$	$\sqrt{(6.5-6)^2 + (6-5)^2} = 1.12$	C_2
$(8, 7)$	$\sqrt{(8-2.5)^2 + (7-2.25)^2} = 7.57$	$\sqrt{(6.5-8)^2 + (6-7)^2} = 1.80$	C_2
$(7, 8)$	$\sqrt{(7-2.5)^2 + (8-2.25)^2} = 7.90$	$\sqrt{(6.5-7)^2 + (6-8)^2} = 2.06$	C_2

Correction

Résultats de l'affectation (Itération 2) :

- Cluster C_1 : $\{(1,1),(2,1),(4,3),(3,4)\}$
- Cluster C_2 : $\{(5,4),(6,5),(8,7),(7,8)\}$

Convergence :

Les clusters n'ont pas changé après cette itération. Les centres ont convergé :

$$C_1 = (2.5, 2.25), \quad C_2 = (6.5, 6)$$

Variantes K-means

1. **K-means++**
   - Améliore l'initialisation des centroïdes
   - Les premiers centroïdes sont choisis pour être éloignés les uns des autres
   - Réduit la sensibilité aux centres initiaux et converge plus rapidement
2. **K-medoids (PAM)**
   - Utilise des points réels du dataset comme centres (médoides)
   - Plus robuste au bruit et aux outliers
   - Algorithmes : PAM (Partitioning Around Medoids), CLARA

Variantes K-means

3. Fuzzy C-means

- Chaque point peut appartenir à **plusieurs clusters** avec différents degrés d'appartenance
- Introduit la notion de **logique floue**

4. K-modes et K-prototypes

- **K-modes** : pour données catégorielles
- **K-prototypes** : pour données mixtes (numériques et catégorielles)

5. Mini-batch K-means

- Utilise des **sous-échantillons** du dataset à chaque itération
- Plus rapide sur de grands datasets
- Qualité des clusters légèrement inférieure

Variantes K-means

6. K-means avec contraintes

- Intègre des contraintes must-link/cannot-link
- Semi-supervisé

7. Spherical K-means

- Optimisé pour les données textuelles (cosine similarity)
- Les centroïdes sont normalisés